

Technical Assessment — AI Agent Devt

You are invited to complete a practical exercise related to the development of modular components for an agentic AI system. Please try to address the tasks you pick to the best of your ability.

Instructions

- Choose 2 of the 3 Assessment options presented to you in this document. Your choice cannot be Options 1 and 3. (The two other combinations are permitted). There are two tasks per Assessment Option.
- You may complete all or only certain aspects of the two options you pick. Naturally the more tasks and aspects you cover the better.
- You have up to 1 week to complete the exercise from the day it was sent to you.
- You may use any tools, libraries, or AI assistants (e.g., coding assistants, LLMs, etc.).
- Use AI tools for coding is fine if your choices are judicious. If you choose to do so for this technical assessment, document which AI assistant you used and share your chat sessions in English with us. We would like to understand how you work.

General expectations

We are interested in:

- Your approach to problem solving
- Your Python code quality and structure
- How you go about documenting your work
- Your testing practices
- Your design decisions

Hint: follow good software development practices as you would in a real project.

Submission

Please provide:

- A public or shared Git based repository (GitHub / GitLab) with your complete code, so we may see your commits' timeline.
- A README explaining:
 - How to run the project
 - Design decisions
 - Tools and technologies used
 - Any AI tools used during development
- Any additional documentation you consider relevant
- On the day of the 2nd technical interview, you may choose to present your work as a code review or with slides and code, or with code and pseudo-code in the proportion of your choice. This is entirely up to you.

Option 1 — Perception + RAG

Design and implement a listening module and a multi-tenant retrieval system with filtering capabilities and minimal latency design choices.

Required tasks

For both tasks, provide (as needed) a simple API (e.g., using FastAPI) or the CLI.

Task 1:

Implement a real-time event ingestion from simulated API:

The signature of seismic events consists in basic location and intensity data as well as the earthquake's timestamp:

- event's ID ('eid'; type int, values in $\mathbb{N}$),
- event's UTC timestamp ('timestamp'; type `datetime.datetime`)
- hypocenter latitude ('lat'; type float, values in $[-90.0^\circ, +90.0^\circ]$),
- hypocenter longitude ('lon'; type float, values in $[-180.0^\circ, +180.0^\circ]$),
- hypocenter depth ('depth'; type float, values in $[-100.0, 0.0]$)
- earthquake's magnitude ('Mw'; type float, values in $[0.0, 9.0]$),

located at

- distance ('dist'; type float, values in $\mathbb{R}^+$), and
- azimuth ('azi'; type float, values in $[-180.0^\circ, +180.0^\circ]$),

from a distant location located on the surface of the earth at

- latitude ('loclat', type float, values in $[-90.0, +90.0]$),
- longitude ('loclon'; type float, values in $[-180.0^\circ, +180.0^\circ]$),

where the seismic event is likely to have been felt could have caused damages.

Design and/or implement LISTENER, a routine to pull real time basic seismic data (as above) every "m" minutes (m is a positive integer variable) as a JSON payload from a distant seismic data Agency "A"'s REST server or, as automatic fall-back, from another distant Agency "B" in case the first agency becomes temporarily unavailable and until Agency A's recovery.

Consider that agencies make new data available as a new event occurs so the time line of past events is immutable.

Check type and validity of downloaded data as needed and log anomalies for later auditing. If the routine must be stopped and reinitiated or switches from Agency A to B and back to A, make sure that data already downloaded is not pulled a second time, as that would produce duplicated data, i.e. data that only differs by its download- timestamp – which is different from the "event's timestamp").

Task 2:

Implement a minimum latency RAG toy-system that:

- ingests a small dataset (e.g., text documents, and CSV/TSV) (You will need to choose segmentation methodologies that are optimal for (a) natural language in terms of expressivity and semantic granularity and (b) for the semi-structured data in CSV/TSV.)
- transforms and indexes the data for retrieval from a vector store (pick an indexing methodology)
- supports querying using natural language or structured queries
- applies filtering based on metadata (e.g., design an incremental filtering by categories $A \subset B \subset C \subset D$, timestamp, source or provenance and an arbitrary combination of the above)
- computes a measure of completeness of the inferred response.

We expect:

- that you provide at least 10 example-queries, where half of them should include at least two distinct topics in the same sentence.
- a demonstration of filtering and gating behavior applied to any relevant aspect and stage of the extraction pipeline.

Option 2 — Belief state (conceptual design + fully justifiable implementation choices)

Required tasks

You want to model and code epistemic uncertainty propagation for an Agentic AI system that includes a BELIEF module. The BELIEF module deals with uncertain data, i.e. data subject to updates and, in that sense, data that is not destined to be ingested and indexed by the RAG system.

In the real world, that module should handle uncertain, multi-variate data, organized as one or more multi-layered heterogeneous arrays (e.g., spatio-temporal or spatio-parametrized tensors dubbed "grids").

The toy example we suggest you scrutinize, involves *uncertain data*, and how to deal with that uncertainty. Your goals are **(1)** to model / represent uncertainty and to store and update the data in an efficient and type-safe way; **(2)** how to implement one or more update mechanisms to handle uncertainty propagation of dynamic data.

We do not ask that you necessarily produce a large quantity of code or at times, for some of the question's aspects below, any code at all. However we ask that you research the subject of the assessment and come up with design choices that you will be expected to present and defend, with regard to code design(s) and code implementation(s).

(Keywords: low latency, safe, oop, complexity, uncertainty)

Efficient storage of data:

The data is the velocity model for propagating P- and S-waves (V_p , V_s in km.s^{-1}) in a particular region of the Earth's upper crust.

"A velocity model represents the distribution and variation of seismic wave speeds within Earth's subsurface. It reflects subsurface properties such as material composition, density, porosity, and temperature. Geophysicists rely on the analysis and interpretation of the velocity model to develop refined models of the subsurface geology, which are essential in resource exploration, earthquake seismology, etc." (Source: Wikipedia)

Mathematically a seismic velocity model is a spatial function (or set of functions) that assigns seismic wave speeds to points in the Earth. $v = V(x,y,z)$ for a full 3D model, where (x,y,z) are spatial coordinates. Additional components may be added to parametrize v , which for us stands for the S-wave velocity, V_s being responsible for seismic shock propagation in the Earth's crust. (The physical details of the phenomenon are not important here). The data model in this case consists of very large CSV files with V_s values

V_s (v_s ; type float, values in $[0.0, 5.0]$)

indexed by ['lat', 'lon', 'depth', 'soilsat'], with type and values:

latitude ('lat'; type float, values in $[-90.0^\circ, +90.0^\circ]$),

longitude ('lon'; type float, values in $[-180.0^\circ, +180.0^\circ]$),

depth ('depth'; type float, values in $[-10.0, 0.0]$), and

soil_saturation ('soilsat'; type float, values in $[0.0, 1.0]$)

Task 1:

Several uncertainties may arise in this case:

- experimental uncertainty about 3D S-wave velocity ('vs') measurements,
- uncertainty about the value of soil saturation ('soilsat') itself,
- uncertainty about the dependency of Vs with respect to soil saturation.

Bear in mind that:

- Physical constraints: Velocities are positive
- Geological reality: Velocities often follow log-normal or gamma distributions due to multiplicative geological processes
- Measurement uncertainty: Tomographic measurements have spatially varying resolution and uncertainty
- Dependency on soil saturation: Geologic properties such as Vs tend to increase as soil saturation increases, due to geological processes, heavy rain, or anthropic effects, such pumping sub-surface liquids in or out. Soil saturation may also vary with time, due to seepage, natural evaporation or any other admissible physical mechanism.

Represent the belief state for seismic S-wave velocities with uncertainty. Do you need statistical moments for an expressive representation ? In a toy-example create relevant implementation in Python.

Task 2:

What update mechanism(s) could you use and what code design(s) could you support for belief state updates, based on the following considerations:

- each code component should have clear responsibilities and be easy to swap/update,
- update mechanisms should be upgradable or extendable, e.g. with new update mechanism(s),
- memory efficient for data with large footprint and efficient GPU compute,
- minimal code change when going from single-user execution to multi-user executions (for instance for concurrent data extraction and updates in several disjoint regions, with execution scalability (minimal I/O latency, memory mapping, ...?)),
- tracking belief reliability though belief state update.
- ease of coding, ease of debugging, latency, compute cost.

Supplement: (only for the wary "traveller"!)

We do recommend that you ponder the two tasks above independently of the code provided in Appendix A, at the end of this document and as a jupyter handbook. We provide the code below only as a non-exhaustive collection of design choices ,including *things you should definitely avoid doing*, in order to respect the constraints and requirements of the project.

Option 3 — Simple reasoning & actuation

Design and implement a multi-tenant pipeline backend, that orchestrates multiple services asynchronously. Here "Reasoning" and "Actuation" consist in automatically selecting and executing one of two downstream toy-services according to some arbitrary heuristic.

One of those two downstream toy-services will execute floating point arithmetic calculations of arbitrary precision, in a Shell of your choice (Python, *nix, etc...). Execution must be triggered by calling a separate service whose output is passed back to the calling service, and to eventually produce a result as output of the pipeline.

Defining the second downstream toy-service is up to you, although we suggest one later in the task descriptions. Regardless it must follow a call protocol similar to that of the first downstream toy-service. For demonstration purposes the second downstream service can be some existing service of your choice, accessible via its API or the processing pipeline described in Task 1.

Required tasks

For both tasks, provide (as needed) a simple API (e.g., using FastAPI) or the CLI. Refrain from using any AI agent framework to build your solution. It would be counterproductive to use any for this test.

Task 1: (same task as for Option 1)

Implement a real-time event ingestion from simulated API:

The signature of seismic events consists in basic location and intensity data as well as the earthquake's timestamp:

- event's ID ('eid'; type int, values in $\mathbb{N}$),
- event's UTC timestamp ('timestamp'; type `datetime.datetime`)
- hypocenter latitude ('lat'; type float, values in $[-90.0^\circ, +90.0^\circ]$),
- hypocenter longitude ('lon'; type float, values in $[-180.0^\circ, +180.0^\circ]$),
- hypocenter depth ('depth'; type float, values in $[-100.0, 0.0]$)
- earthquake's magnitude ('Mw'; type float, values in $[0.0, 9.0]$),

located at

- distance ('dist'; type float, values in $\mathbb{R}^+$), and
- azimuth ('azi'; type float, values in $[-180.0^\circ, +180.0^\circ]$),

from a distant location located on the surface of the earth at

- latitude ('loclat', type float, values in $[-90.0, +90.0]$),
- longitude ('loclon'; type float, values in $[-180.0^\circ, +180.0^\circ]$),

where the seismic event is likely to have been felt could have caused damages.

Design and/or implement LISTENER, a routine to pull real time basic seismic data (as above) every "m" minutes (m is a positive integer variable) as a JSON payload from a distant seismic data Agency "A"'s REST server or, as automatic fall-back, from another distant Agency "B" in case the first agency becomes temporarily unavailable and until Agency A's recovery.

Consider that agencies make new data available as a new event occurs so the time line of past events is immutable.

Check type and validity of downloaded data as needed and log anomalies for later auditing. If the routine must be stopped and reinitiated or switches from Agency A to B and back to A, make sure that data already downloaded is not pulled a second time, as that would produce duplicated data, i.e. data that only differs by its download- timestamp – which is different from the "event's timestamp").

Task 2:

- Locally implement a multi-tenant, system that:
 - Receives requests (e.g., inputs or API calls from one or more users),
 - Triggers a choice of processing steps or downstream pipelines as a function of the received request.
You can use the LISTENER of Task 1, as one such downstream pipeline, by triggering an immediate data pull instead of waiting for the next automatic pull.
Implement at least 2 such distinct pipelines, where the second could be to trigger a small arithmetic calculation, performed by an external tool, based on the user's arbitrary request.
 - Passes outputs from one pipeline to the next (process-chaining),
 - Handles errors or failures gracefully,
 - Minimizes latency,
 - Exposes execution and wall-clock times for each request.
- Provide:
 - A simple orchestration mechanism
 - Example services.
Services can be mocked, except for the two first ones, that should be minimum viable prototypes)
 - A demonstration of execution flow

Appendix A

WARNING: This mockup contains misleading or non-optimal design choices for the assessment under Option 2.

```
import pandas as pd
import numpy as np
from typing import Dict, List, Tuple, Any
import pickle
import json
from datetime import datetime
import threading
import time

class BeliefState:
    """
    Docstring
    """
    def __init__(self):
        self.data = pd.DataFrame(columns=['lat', 'lon', 'depth', 'soilsat', 'vs',
        'vs_uncertainty'])
        self.metadata = {}

    def add_measurement(self, lat: float, lon: float, depth: float, soilsat: float, vs: float,
    uncertainty: float):
        new_row = {
            'lat': lat, 'lon': lon, 'depth': depth, 'soilsat': soilsat,
            'vs': vs, 'vs_uncertainty': uncertainty
        }
        self.data = pd.concat([self.data, pd.DataFrame([new_row])], ignore_index=True)

    def get_belief_at_location(self, lat: float, lon: float, depth: float, soilsat: float) ->
    Dict[str, Any]:
        mask = (
            (self.data['lat'] == lat) &
            (self.data['lon'] == lon) &
            (self.data['depth'] == depth) &
            (self.data['soilsat'] == soilsat)
        )
        result = self.data[mask]
        if len(result) > 0:
            return result.iloc[0].to_dict()
        return {}

class Updater:
    """
    Docstring
    """
    def __init__(self):
        self.update_count = 0

    def update_belief(self, current_belief: Dict, new_observation: Dict) -> Dict:
        updated_vs = (current_belief.get('vs', 0) + new_observation['vs']) / 2
        updated_uncertainty = (current_belief.get('vs_uncertainty', 1) +
        new_observation['uncertainty']) / 2

        return {
            'vs': updated_vs,
            'vs_uncertainty': updated_uncertainty,
            'timestamp': datetime.now().isoformat()
        }
```



```

class GlobalBeliefManager:
    """
    Docstring
    """
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        if not hasattr(self, 'initialized'):
            self.belief_states = {} # Maps tenant_id to belief state
            self.initialized = True

    def get_belief_state(self, tenant_id: str):
        if tenant_id not in self.belief_states:
            self.belief_states[tenant_id] = BeliefState()
        return self.belief_states[tenant_id]

class FilePersistence:
    """
    Docstring
    """
    def save_to_file(self, data: Any, filename: str):
        with open(filename, 'wb') as f:
            pickle.dump(data, f)

    def load_from_file(self, filename: str) -> Any:
        with open(filename, 'rb') as f:
            return pickle.load(f)

class SequentialProcessor:
    """
    Docstring
    """
    def __init__(self, updater: Updater):
        self.updater = updater

    def process_batch(self, measurements: List[Dict]):
        results = []
        for measurement in measurements:
            result = self.updater.update_belief({}, measurement)
            results.append(result)
        return results

def calculate_simple_stats(data_list: List[float]) -> Dict[str, float]:
    """
    Docstring
    """
    return {
        'mean': np.mean(data_list),
        'std': np.std(data_list),
    }

def example_usage():
    """
    Docstring

```

```
"""
manager = GlobalBeliefManager()
belief_state = manager.get_belief_state("tenant_1")

# Add some measurements
belief_state.add_measurement(40.7, -74.0, -2.0, 0.3, 2.5, 0.1)
belief_state.add_measurement(40.7, -74.0, -2.0, 0.4, 2.7, 0.15)

# Get belief
belief = belief_state.get_belief_at_location(40.7, -74.0, -2.0, 0.3)
print(f"Belief: {belief}")

if __name__ == "__main__":
    example_usage()
```